

Lov pokladů

Časový limit: 8 s Limit paměti: 256 MiB

Indiana Jones hledá poklady na dlouho zapomenutém ostrově pomocí magického kompasu. Ostrov si můžeme představit jak obrovskou mřížku $N \times N$ a na $K \leq 3$ jejích políčkách se nachází truhly s pokladem. Váš úkol je jednoduchý: Pomožte Indiana Jonesovi je najít všechny!

Když položíte kompas na jedno z políček, tak nejprve najde nejkratší cesty k truhle pomocí pohybů nahoru, dolů, doleva a doprava (tedy nejbližší truhlici podle Manhattanské vzdálenosti) a poté vám ukáže směr prvního kroku. Pokud existuje více prvních kroků, které vedou k nejbližší truhle (nebo k více nejbližším truhlám), kompas vám ukáže všechny takové kroky. Pokud políčko obsahuje truhlu, tak vám to kompas ukáže místo prvních kroků.

Kdykoliv najdete truhlu, vezmete si z ní poklad, ale truhla zůstane stát tam, kde byla – je příliš těžká. Váš kompas neví, které truhly jsou plné a bude ukazovat k nejbližší truhle, ať už je plná nebo prázdná.

Zkuste najít pozice všech truhel na co nejméně položení kompasu.

Úloha

Tato úloha je interaktivní. Pro každý test case (tedy spuštění vašeho programu) váš program musí provést několik lovů pokladů. S graderem byste měli komunikovat za použití knihovny od organizátorů. Knihovna obsahuje tyto deklarace:

- **void** *NextHunt*(**int** &*N*, **int** &*K*) — Pro začátek dalšího lovu pokladů zavolejte tuto funkci. Do svých argumentů nastaví velikost mřížky *N* a počet pokladů *K*. V případě, že už nezbývají žádné další lovy pokladů, funkce nastaví *N* i *K* na -1 . V tomto případě má váš program hned skončit s návratovým kódem 0. Poznamenejme, že tuto funkci můžete zavolat i před tím, než najdete všechny truhly. (Například pokud se snažíte pouze získat částečné body.)
- **enum** { *TREASURE* = 0, *DIR_RIGHT* = 1, *DIR_UP* = 2, *DIR_LEFT* = 4, *DIR_DOWN* = 8 }; — Jedná se o konstanty používané v návratových hodnotách funkce *Query* (viz níže).
- **int** *Query*(**int** *x*, **int** *y*) — Pokud políčko na pozici (*x*, *y*) obsahuje poklad, tato funkce vrátí *TREASURE*. Jinak vrátí součet jedné nebo více konstant *DIR_RIGHT*, *DIR_UP*, *DIR_LEFT*, a *DIR_DOWN* udávající, které kroky ukáže kompas položený na políčku (*x*, *y*). Souřadnice *x* a *y* musí být celá čísla mezi 0 a $N - 1$. (Pozn.: V této úloze se souřadnice *y* zvyšuje zhora dolů).

Potom, co *NextHunt* nastaví $N = K = -1$, váš program nesmí zavolat *NextHunt* ani *Query*. Dále nesmí zavolat *Query* před prvním zavoláním *NextHunt*. Pokud váš program tato omezení poruší, nebo provede více jak 1000 dotazů během jednoho lovu pokladů, nebo vrátí hodnoty *x* a/nebo *y* mimo rozsah při zavolání *Query*, tak vám knihovna ukončí program a dostanete run-time-error (RTE) verdikt pro tento test case.

Poklad se považuje za nalezený, pokud jste se dotázali na jeho políčko alespoň jednou. Pokud váš program zavolá *NextHunt* před nalezením všech truhel, tak nespadne s běhovou chybou, ale zohlední se to do vašich bodů (o bodování více níže).

Pro použití knihovny by váš program měl includovat hlavičkový soubor `treasurehuntlib.h`:

```
#include "treasurehuntlib.h"
```

Tento hlavičkový soubor si můžete stáhnout zde: `treasurehuntlib.h`.

Pro zjednodušení psaní vašich řešení si můžete stáhnout jednoduchou implementaci knihovny zde: `treasurehuntlib-public.cpp`. Abyste ji zkompilovali s vaším programem, přidejte její jméno jako parametr překladači, např.:

```
g++ foo.cpp treasurehuntlib-public.cpp
```

kde `foo.cpp` je jméno souboru obsahujícího vaše řešení.

Implementace v `treasurehuntlib-public.cpp` kromě výše zmíněných deklarací podporuje další funkci `void InitFromFile(const char *fileName)`, která použije seznam lovů pokladů ze souboru místo generování vlastních náhodných lovů pokladů. Více detailů najdete v samotném `treasurehuntlib-public.cpp`.

Na vyhodnocovacím serveru bude použita jiná implementace knihovny, takže byste neměli předpokládat nic o jejím chování. Ale můžete předpokládat, že do globální namespace nepřidává žádné deklarace kromě výše zmíněných (*NextHunt*, *Query* a pět enumových konstant).

Váš kód nesmí číst ze standardního vstupu nebo vypisovat na standardní výstup, protože ty budou použity implementací knihovny na vyhodnocovacím serveru pro komunikaci se zbytkem vyhodnocovacího prostředí.

Vstup

Omezení

- $2 \leq N \leq 10^6$
- $1 \leq K \leq 3$
- Během kteréhokoliv jednoho spuštění vašeho programu bude nejvýše 100 000 lovů pokladů.
- Během kteréhokoliv jednoho lovu pokladů můžete provést nejvýše 1000 dotazů.
- Vyhodnocovací systém není adaptivní.

Podúlohy

- Podúloha 1 (10 bodů) $K = 1$
- Podúloha 2 (30 bodů) $K = 2$
- Podúloha 3 (60 bodů) $K = 3$.

Bodování

Podúloha se může skládat z několika test casů (spuštění vašeho programu) a každý test case se může skládat z několika lovů pokladů. Pro účely bodování, všechny lovy pokladů v jedné podúloze jsou vyhodnoceny spolu neohledně na to, ve kterých test casech se nacházely. Pro i -tý lov pokladů označme velikost mřížky $N_i \times N_i$, počet pokladů K_i , počet dotazů vašeho programu Q_i a počet vámi nalezených pokladů F_i . Dále označme S jako celkový počet bodů, který lze získat z této podúlohy. Potom počet bodů, které dostanete za tuto podúlohu je:

- Pokud váš program vždy našel všechny truhly (tedy $F_i = K_i$ pro všechna i), váš počet bodů závisí na $t_i = \frac{Q_i}{\lceil \log_2 N_i \rceil}$:

$$\frac{S}{2} + \frac{S}{2} \cdot \min_i f(t_i) \text{ bodů, kde } f(t_i) = \begin{cases} 1, & t_i \leq 11 \\ 1 - (t_i - 11)/9, & 11 \leq t_i \leq 20 \\ 0, & t \geq 20. \end{cases}$$

- Pokud váš program někdy nenašel všechny truhly (tedy existuje i , takové že $F_i < K_i$), tak dostane

$$\frac{S}{2} \cdot \min_i \frac{F_i}{K_i} \text{ bodů.}$$

Jinak řečeno, jednu polovinu bodů dostanete za nalezení všech truhel a druhou polovinu za jejich nalezení na co nejméně dotazů. Pro plný počet bodů musí vaše řešení najít všechny truhly v nejvýše $11 \lceil \log_2 N_i \rceil$ dotazech. Mezi $11 \lceil \log_2 N_i \rceil$ a $20 \lceil \log_2 N_i \rceil$ dotazy se počet bodů snižuje lineárně. A pokud vaše řešení provede více jak $20 \lceil \log_2 N_i \rceil$ dotazů, dostane pouze první polovinu bodů za nalezení všech truhel. (Symboly $\lceil \cdot \rceil$ znamenají, že hodnota $\log_2 N_i$ se zaokrouhlí nahoru na nejbližší celé číslo.)

Pokud by vám formule výše dala neceločíselný počet bodů za tuto podúlohu, váš počet bodů se zaokrouhlí k nejbližšímu celému číslu.

Pokud váš program dostane runtime error nebo nedodrží výše zmíněný protokol na používání knihovny, dostane 0 bodů za celou podúlohu. Proto, abyste dostali částečné body za nalezení části pokladů, váš program musí skončit zavoláním *NextHunt*.

Příklad

Volání	Návratová hodnota
NextHunt(N, K)	$N = 4, K = 1$
Query(2, 0)	$DIR_DOWN + DIR_RIGHT = 9$
Query(3, 1)	DIR_DOWN
Query(3, 2)	$TREASURE$
NextHunt(N, K)	$N = -1, K = -1$